



QUANTPHEMES RL

Q-Learning Paper-Trading Fleet

A reproducible research, diagnostics, and live deployment system for Hong Kong ETF trading strategies.

WHY THIS PROJECT EXISTS

One workflow from hypothesis to paper trading

Research

Run repeatable historical experiments across ETF assets, intervals, features, rewards, and agents.

Explain

Use reports, oracle checks, coverage, and fee diagnostics to understand why the model behaves a certain way.

Deploy

Promote selected `q_state.pkl` artifacts to paper trading through the Quantphemes API.

CURRENT STATUS

Seven-bot paper fleet integrated in one repo

BOTS

7

Five selected strategies plus two collaborative 1-hour imports.

ASSETS

3

2800.HK, 2828.HK, and 7226.HK.

INTERVALS

30m/1h

Five internal paper models are 30-minute; two collaborative models are 1-hour.

CORE TESTS

85

Full suite passes with lint and docs build.

DEPLOYMENT

Azure

Systemd services with JSON decision logs.

BOT PORTFOLIO

Different hypotheses run side by side

BOT	ASSET	INTERVAL	REWARD / ENCODER	ACTIONS	STATUS
RL_CROSS	7226.HK	30m	drawdown / three_feature	cash/full	paper running
RL_VOL	7226.HK	30m	drawdown / volatility	cash/full	paper running
RL_FACOV	7226.HK	30m	fee-aware / z-score	cash/half/full	watch fees
RL_BROAD_A	2800.HK	30m	fee-aware / z-score	cash/half/full	paper running
RL_BROAD_B	2828.HK	30m	log-return / volatility	cash/half/full	paper running
Collaborative HSI 1H	2800.HK	1h	QP-spec / three_feature	cash/full	OOS -4.77%
Collaborative Leverage 1H	7226.HK	1h	QP-spec / three_feature	cash/full	OOS -3.12%

BOT INTERPRETATION

What each live candidate is meant to teach us

BOT	WHAT IT TEACHES US
RL_CROSS	Simple 30m 7226.HK leveraged baseline: compact state, binary action, drawdown control.
RL_VOL	Tests whether 7226.HK decisions improve when the policy knows the current volatility regime.
RL_FACOV	Ternary 7226.HK model with fee-aware reward; useful for position sizing but watched for turnover.
RL_BROAD_A	30m 2800.HK broad benchmark that shows behavior on lower-volatility HSI exposure.
RL_BROAD_B	30m 2828.HK broad/federated comparison using volatility-aware state and ternary actions.
Collaborative HSI 1H	External 1h 2800.HK benchmark; OOS net -4.77% over 750 days, alpha -6.46 pp vs B&H.
Collaborative Leverage 1H	External 1h 7226.HK benchmark; OOS net -3.12% over 450 days, alpha -3.56 pp vs B&H.

COLLABORATIVE 1H TEST RESULTS

Different windows, useful external benchmarks

STRATEGY	PERIOD	DAYS	NET RETURN	FEES	ROUND TRIPS	WIN RATE	ALPHA VS B&H
Collaborative HSI 1H	2022-05-13 → 2025-06-09	750	-4.77%	HKD 53,801	491	24.24%	-6.46 pp
Collaborative Leverage 1H	2024-04-05 → 2026-02-06	450	-3.12%	HKD 29,372	288	15.97%	-3.56 pp

These are not same-window comparisons with the 30-minute grid. They show that the 1-hour imported policies are compatible, but their own out-of-sample tests were net negative after fees.

LEVERAGED ETF RATIONALE

Why more paper slots use 7226.HK

Fee hurdle

The round trip cost is about 40 bps. Many 2800.HK intraday moves are smaller than that, so cash can be a rational learned action.

Research signal

Leveraged ETFs can move more intraday, giving the Q-table more room to learn whether exposure is worth the fee.

2800.HK remains the broad HSI benchmark. Leveraged ETFs are used more heavily for paper observation because their larger movement makes strategy behavior easier to distinguish.

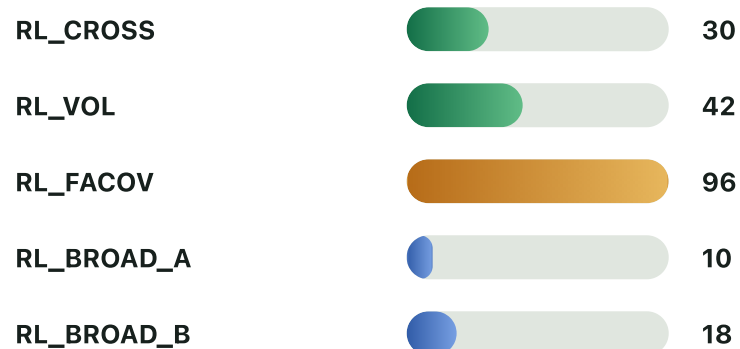
BACKTEST SNAPSHOT

Selected models beat buy-and-hold in the test window after fees

Alpha vs B&H



Trade Count



ARCHITECTURE

Shared core, thin applications

Config

YAML plus Pydantic validation define the whole experiment or production bot.

Plugins

Data source, encoder, reward, and agent are selected through a registry, not hard-coded branches.

Core

TradingEnv, Portfolio, StateEncoder, RewardFunction, and Agent are shared by training and live execution.

Flow: YAML config → registry → data/state/reward/agent → training report + Q-table → live bot → Quantphemes API.

DATA AND CONFIGURATION

Strategy behavior is declared, validated, and reproducible

Data sources

The current research data mainly comes from Webull and Futu exports/downloads, normalized into HK-time bars.

YAML configs

Asset, date range, fee, decision times, encoder, reward, agent, and artifacts are visible in one file.

Pydantic validation

Bad config or malformed data fails early instead of silently entering training or live execution.

Supported adapters

Bloomberg XLSX is supported for future exports, but it was not the main source for the current paper fleet.

YAML REFERENCE

A strategy is visible in one config file

```
name: production_rl_broad_a
data:
  source: csv
  path: data/raw/2800_hk_30m_webull_futu.csv
  symbol: 2800.HK
market:
  decision_times: ["10:00", "10:30", "11:00", "11:30", "13:30", "14:00", "14:30", "15:00", "15:30"]
  fee_bps_one_side: 20
state:
  encoder: zscore_bins
reward:
  function: fee_aware
agent:
  type: qtable
  kwargs:
    num_actions: 3
```

PLUGIN REGISTRY

New strategy ideas do not require rewriting the runner

Data

`csv`, `futu`,
`bloomberg`

State

`three_feature`,
`zscore_bins`,
`with_volatility`,
`with_rsi`

Reward

`log_return`,
`fee_aware`,
`drawdown_penalty`

Agent

`qtable`, baselines, and
future model plugins.

The runner asks the registry to build the selected components. This keeps the production path consistent as the research space grows.

TRAINING LOOP

How a Q-table is produced

Walk-forward training

Historical days are split into training and test windows. Each training episode shuffles days and starts each day with a fresh portfolio.

Decision steps

At each decision time, the environment measures NAV before trade, executes action, applies fees, advances time, and computes reward.

Q update

The agent updates one cell: decision point, state, action. Visit counts determine the per-cell learning rate.

Artifact output

The selected result folder contains `q_state.pkl`, config snapshot, metrics, logs, and HTML diagnostics.

LIVE RUNTIME

How one paper decision is made

1. Fetch current price and broker cash/quantity.
2. Build market context: current price, open, previous close, previous decision price.
3. Encode context into a discrete state.
4. Read the Q-table row for this decision point and choose the greedy action.
5. Convert action into target quantity using lot size and available cash.
6. Patch Quantphemes holding and write one structured JSON log line.

COLLABORATIVE MODEL INTEGRATION

External Q-tables now run through the same code path

Compatibility

Both collaborative tables are `5 × 27 × 2` with visit counts and the same three-feature threshold structure.

Conversion

Their standalone pickle files were converted into our standard `QTableAgent.save` artifact format.

Production configs

Two YAMLs define asset, decision times, thresholds, artifact paths, and runtime log locations.

Bug fix

The live bot now uses previous decision price for `delta_prev`, not the current price.

FRAMEWORK CHOICES

Why these tools?

Python, NumPy, Pandas

Best fit for market data processing, vectorized metrics, and quick research iteration.

Pydantic + YAML

Readable experiment files with strict validation and inheritance.

Pytest + Ruff

Fast regression testing and style checks before deployment.

MkDocs Material

Searchable, polished documentation for mixed technical and non-technical readers.

TRADING RULES

Financial assumptions are explicit and tested

20 bps

One-side transaction fee, applied only when position changes.

EOD

Live bot targets flat quantity near the close; each training day starts fresh.

Lot size

Target quantity respects exchange lot constraints and available cash.

DIAGNOSTICS

We diagnose the model, not only the final NAV

Coverage

Which decision/state/action cells were actually visited during training.

Oracle

Perfect-foresight upper bound under the same fee model.

Signal Checks

Feature bins, conditional returns, and mutual information.

LIVE OPERATIONS

Production-style deployment is observable and reversible

Server

Azure VM, Python virtual environment, one systemd service per bot, restart policy enabled.

Logs

One JSON line per decision: price, state tuple, action, target quantity, current quantity, and status.

Secrets

API key and portfolio/strategy IDs are in server-side `.env`, never committed.

Rollback

Stop a service, restore a previous Q-table, restart only that bot.

QUANTPHEMES API INTEGRATION

Broker behavior shaped the deployment design

- Documented endpoints are wrapped in `api_client/quantphemes.py` .
- Undocumented price endpoints are isolated in `api_client/_undocumented.py` .
- Holding PATCH calls are tested through a sync helper with mocked network tests.
- Symbol tradeability is validated separately from price availability.

QUALITY CONTROLS

Trading invariants are protected by tests

Core tests

Portfolio fees, environment reward timing, force-close behavior, Q-table updates, baselines, and registry loading.

Live bot tests

Target quantity sizing, dry-run safety, heartbeat behavior, runtime state recovery, and previous decision price state.

Experiment tests

Config inheritance, end-to-end smoke run, report rendering, oracle, signal diagnostics, and comparison tool.

CI gates

```
ruff check ., pytest -q, and mkdocs build --strict .
```

DOCUMENTATION

The project has a public knowledge base

GitHub

github.com/YelamanKarassay/Algo-Trading-RL

Docs Portal

yelamankarassay.github.io/Algo-Trading-RL/

The docs cover architecture, data requirements, experiment workflow, live runbook, API notes, research notes, and developer extension rules.

CURRENT BOUNDARIES

What the report does and does not claim

It claims

The research-to-paper pipeline is implemented, tested, documented, and able to run multiple Q-table strategies.

It does not claim

The current paper strategies are production-profitable. Live observation and broker behavior still matter.

Backtests include fees, but they should be read as candidate filters and diagnostics, not final investment evidence.

DISCUSSION POINTS FOR QUANTPHEMES

Platform clarifications that affect paper operation

- Which HK ETFs are tradeable through paper and live broker routes?
- How should we verify that a paper PATCH produced a visible order or holding change?
- What are the exact strategy automation limits under the current plan?
- Can Quantphemes provide clearer intraday historical endpoint access or export formats?

CLOSING

We have a working research-to-paper pipeline

The project is ready for a serious technical discussion: what the models do, how the system is built, how paper decisions are generated, and where Quantphemes platform behavior shapes deployment.